

Deep Generative Models Background

Fall Semester 2026

René Vidal

Director of the Center for Innovation in Data Engineering and Science (IDEAS),
Rachleff University Professor, University of Pennsylvania
Amazon Scholar & Chief Scientist at NORCE

Outline

- **Basics of Probability, Statistics, Information Theory**
 - Discrete and Continuous Distributions, Independence
 - Marginals, Conditionals, Product Rule, Bayes Rule & Examples for Gaussians
 - Expectations, Covariance, Entropy, KL Divergence, Mutual Information
- **Basics of Linear Algebra**
 - Matrix, eigen-vectors, eigen-values, rank, eigen-value decomposition
 - Matrix properties: inverse, trace, determinant, positive definiteness
 - Singular value decomposition (SVD), compact SVD
 - SVD properties: inverse, trace, determinant
- **Basics of Optimization**
 - Gradient, Hessian, first and second order conditions for minima/maxima,
 - Gradient and stochastic gradient descent, alternating minimization, Lagrange Multipliers
 - Matrix-valued derivatives, Least squares,

Review of Probability and Statistics

- Data $\mathbf{x} \in \mathcal{X}$ follows some data distribution $\mathbf{x} \sim p_{\theta}(\mathbf{x})$ with parameter θ .
 - Properties: $p_{\theta}(\mathbf{x}) \geq 0$ (non-negativity), $\int p_{\theta}(\mathbf{x}) d\mathbf{x} = 1$ (add up to 1)
- **Continuous case:** $\mathbf{x} \in \mathbb{R}^D$, $p_{\theta}(\mathbf{x})$ is a probability density function.
 - E.g., Gaussian distribution: $\mathbf{x} \sim \mathcal{N}(\mathbf{x} \mid \boldsymbol{\mu}, \boldsymbol{\Sigma})$, $\theta = (\boldsymbol{\mu}, \boldsymbol{\Sigma})$, $\boldsymbol{\mu} \in \mathbb{R}^D$, $\boldsymbol{\Sigma} \in \mathbb{R}^{D \times D}$, $\boldsymbol{\Sigma} \succcurlyeq 0$

$$p_{\theta}(\mathbf{x}) = (2\pi)^{-\frac{D}{2}} \det(\boldsymbol{\Sigma})^{-\frac{1}{2}} \exp\left(-\frac{1}{2}(\mathbf{x} - \boldsymbol{\mu})^{\top} \boldsymbol{\Sigma}^{-1}(\mathbf{x} - \boldsymbol{\mu})\right)$$

- **Discrete case:** $\mathbf{x} \in \{\mathbf{x}_1, \dots, \mathbf{x}_K\}$, $p_{\theta}(\mathbf{x})$ is a probability mass function.
 - E.g., Categorical distribution: $\mathbf{x} \sim \text{Cat}(\mathbf{x} \mid \boldsymbol{\pi})$, $\theta = \boldsymbol{\pi}$, $\pi_k \geq 0$, $\sum_k \pi_k = 1$

$$p_{\theta}(\mathbf{x} = \mathbf{x}_k) = \pi_k$$

- **Independence:** $\mathbf{x}$ and $\mathbf{y}$ are independent if and only if $p(\mathbf{x}, \mathbf{y}) = p(\mathbf{x})p(\mathbf{y})$.

Marginals, Conditionals, Product Rule, Bayes Rule

- **Marginal distribution**

- Continuous case:

$$p(\mathbf{x}) = \int p(\mathbf{x}, \mathbf{y}) d\mathbf{y}$$

- Discrete case:

$$p(\mathbf{x}) = \sum_{\mathbf{y}} p(\mathbf{x}, \mathbf{y})$$

- **Conditional distribution**

$$p(\mathbf{y} \mid \mathbf{x}) = \frac{p(\mathbf{x}, \mathbf{y})}{p(\mathbf{x})}$$

- **Product rule**

$$p(\mathbf{x}, \mathbf{y}) = p(\mathbf{x} \mid \mathbf{y})p(\mathbf{y}) = p(\mathbf{y} \mid \mathbf{x})p(\mathbf{x})$$

- **Bayes rule**

$$p(\mathbf{y} \mid \mathbf{x}) = \frac{p(\mathbf{x} \mid \mathbf{y})p(\mathbf{y})}{p(\mathbf{x})}$$

Example: Marginal and Conditional for a Gaussian

- Assume $\mathbf{x} \sim \mathcal{N}(\mathbf{x} \mid \boldsymbol{\mu}, \boldsymbol{\Sigma})$, where

$$\mathbf{x} = \begin{bmatrix} \mathbf{x}_a \\ \mathbf{x}_b \end{bmatrix}, \quad \boldsymbol{\mu} = \begin{bmatrix} \boldsymbol{\mu}_a \\ \boldsymbol{\mu}_b \end{bmatrix}, \quad \boldsymbol{\Sigma} = \begin{bmatrix} \boldsymbol{\Sigma}_a & \boldsymbol{\Sigma}_{ab} \\ \boldsymbol{\Sigma}_{ab}^\top & \boldsymbol{\Sigma}_b \end{bmatrix}$$

- Then, we get the following results

$$p(\mathbf{x}_a) = \mathcal{N}(\mathbf{x}_a \mid \boldsymbol{\mu}_a, \boldsymbol{\Sigma}_a),$$
$$p(\mathbf{x}_a \mid \mathbf{x}_b) = \mathcal{N}(\mathbf{x}_a \mid \hat{\boldsymbol{\mu}}_a, \hat{\boldsymbol{\Sigma}}_a),$$

where

$$\hat{\boldsymbol{\mu}}_a = \boldsymbol{\mu}_a + \boldsymbol{\Sigma}_{ab} \boldsymbol{\Sigma}_b^{-1} (\mathbf{x}_b - \boldsymbol{\mu}_b)$$
$$\hat{\boldsymbol{\Sigma}}_a = \boldsymbol{\Sigma}_a - \boldsymbol{\Sigma}_{ab} \boldsymbol{\Sigma}_b^{-1} \boldsymbol{\Sigma}_{ab}^\top$$

Warm-up exercise -> HW1

Example: Marginal for a Mixture of Gaussians

- Assume $\mathbf{y} \sim \text{Cat}(\mathbf{y} \mid \boldsymbol{\pi})$, $\mathbf{y} \in \{1, \dots, K\}$.
- Assume $\mathbf{x} \mid \mathbf{y} \sim \mathcal{N}(\mathbf{x} \mid \boldsymbol{\mu}_{\mathbf{y}}, \boldsymbol{\Sigma}_{\mathbf{y}})$.
- Then, $\mathbf{x}$ is a mixture of Gaussians

$$\begin{aligned} p(\mathbf{x}) &= \sum_{\mathbf{y}} p(\mathbf{x}, \mathbf{y}) &= \sum_{\mathbf{y}} p(\mathbf{x} \mid \mathbf{y}) p(\mathbf{y}) &= \sum_{k=1}^K p(\mathbf{x} \mid \mathbf{y} = k) p(\mathbf{y} = k) \\ \text{Marginalization} && \text{Product rule} && = \sum_{k=1}^K \mathcal{N}(\mathbf{x} \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k) \pi_k \end{aligned}$$

Expectations and Covariance

- **Expectation**

- Continuous case: $\boldsymbol{\mu}_x = \mathbb{E}[\boldsymbol{x}] = \int \boldsymbol{x} p(\boldsymbol{x}) d\boldsymbol{x}$

- Discrete case: $\boldsymbol{\mu}_x = \mathbb{E}[\boldsymbol{x}] = \sum_k \boldsymbol{x}_k p(\boldsymbol{x} = \boldsymbol{x}_k)$

- **Covariance**

- Continuous case:

- $\boldsymbol{\Sigma}_x = \mathbb{V}[\boldsymbol{x}] = \int (\boldsymbol{x} - \boldsymbol{\mu}_x)(\boldsymbol{x} - \boldsymbol{\mu}_x)^\top p(\boldsymbol{x}) d\boldsymbol{x}$

- $\boldsymbol{\Sigma}_{xy} = \text{Cov}[\boldsymbol{x}, \boldsymbol{y}] = \int (\boldsymbol{x} - \boldsymbol{\mu}_x)(\boldsymbol{y} - \boldsymbol{\mu}_y)^\top p(\boldsymbol{x}, \boldsymbol{y}) d\boldsymbol{x} d\boldsymbol{y}$

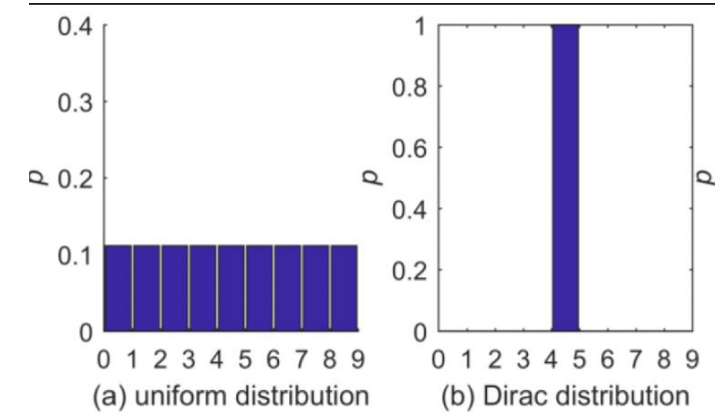
- Discrete case:

- $\boldsymbol{\Sigma}_x = \mathbb{V}[\boldsymbol{x}] = \sum_k (\boldsymbol{x}_k - \boldsymbol{\mu}_x)(\boldsymbol{x}_k - \boldsymbol{\mu}_x)^\top p(\boldsymbol{x} = \boldsymbol{x}_k)$

- $\boldsymbol{\Sigma}_{xy} = \text{Cov}[\boldsymbol{x}, \boldsymbol{y}] = \sum_k (\boldsymbol{x}_k - \boldsymbol{\mu}_x)(\boldsymbol{y}_k - \boldsymbol{\mu}_y)^\top p(\boldsymbol{x} = \boldsymbol{x}_k, \boldsymbol{y} = \boldsymbol{y}_k)$

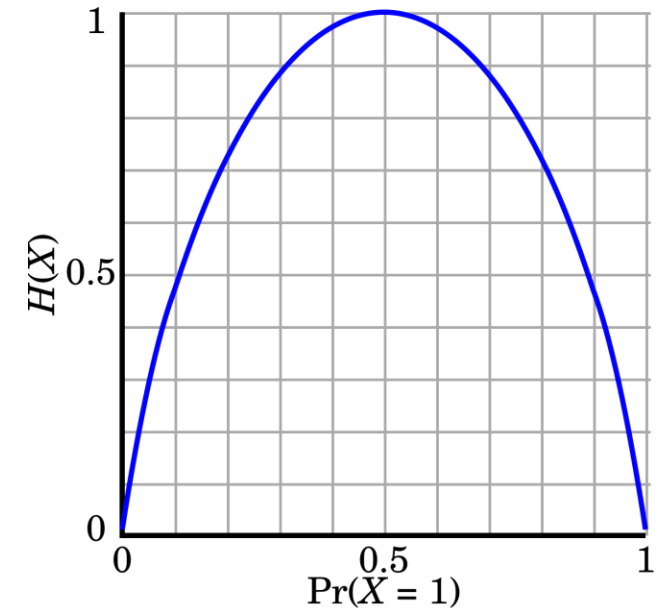
Entropy and Conditional Entropy

- **Entropy** of a random variable $\mathbf{x}$
 - It captures how much “uncertainty” is present in $\mathbf{x}$.
 - **Definition:** $H(\mathbf{x}) = \mathbb{E}_{\mathbf{x} \sim p(\mathbf{x})}[-\log p(\mathbf{x})]$
 - **Continuous:** $H(\mathbf{x}) = -\int_{\mathbf{x}} \log(p(\mathbf{x})) p(\mathbf{x}) d\mathbf{x}$
 - **Discrete:** $H(\mathbf{x}) = -\sum_k \log(\pi_k) \pi_k$ where $\pi_k = P(\mathbf{x} = \mathbf{x}_k)$
- **Examples:**
 - **Uniform:** $\pi_k = \frac{1}{K} \Rightarrow H(\mathbf{x}) = -\sum_k \log(\frac{1}{K}) \frac{1}{K} = \log(K)$
 - **Bernoulli:** $\pi_1 = q \Rightarrow H(\mathbf{x}) = -q \log q - (1 - q) \log(1 - q)$
- **Conditional entropy:** uncertainty of $\mathbf{x}$ when $\mathbf{y}$ is observed
 - $H(\mathbf{x} \mid \mathbf{y}) = \mathbb{E}_{\mathbf{x}, \mathbf{y} \sim p(\mathbf{x}, \mathbf{y})}[-\log p(\mathbf{x} \mid \mathbf{y})]$



High entropy

Low entropy



Entropy of a Bernoulli variable

Kullback–Leibler Divergence and Mutual Information

- **KL divergence** measures the similarity between two distributions p, q

$$\text{KL}[p \parallel q] = \mathbb{E}_{x \sim p} \left[\log \frac{p(x)}{q(x)} \right]$$

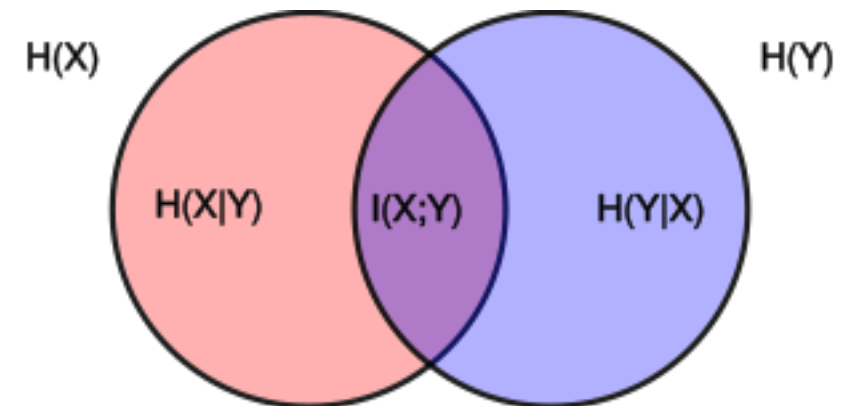
- Non-negativity $\text{KL}[p \parallel q] \geq 0$. Equality holds iff $p = q$.
- In general triangle inequality and symmetry does not hold.

- **Mutual Information** measures the mutual dependence between x and y

$$I(x; y) = \text{KL}[p(x, y) \parallel p(x)p(y)] = \mathbb{E}_{x, y \sim p(x, y)} \left[\log \left(\frac{p(x, y)}{p(x)p(y)} \right) \right]$$

- If x, y are independent, then $I(x; y) = 0$.
- $I(x; y)$ measures the uncertainty in x after observing y .

$$I(x; y) = H(x) - H(x | y) = H(y) - H(y | x)$$



Outline

- Basics of Probability, Statistics, Information Theory
 - Discrete and Continuous Distributions, Independence
 - Marginals, Conditionals, Product Rule, Bayes Rule & Examples for Gaussians
 - Expectations, Covariance, Entropy, KL Divergence, Mutual Information
- **Basics of Linear Algebra**
 - Matrix, eigen-vectors, eigen-values, rank, eigen-value decomposition
 - Matrix properties: inverse, trace, determinant, positive definiteness
 - Singular value decomposition (SVD), compact SVD
 - SVD properties: inverse, trace, determinant
- Basics of Optimization
 - Gradient, Hessian, first and second order conditions for minima/maxima,
 - Gradient and stochastic gradient descent, alternating minimization, Lagrange Multipliers
 - Matrix-valued derivatives, Least squares,

Eigenvectors, Eigenvalues, Rank, E-Decomposition

- $(\lambda, v) \in \mathbb{C} \times \mathbb{C}^n$ is an **eigenvalue-eigenvector** pair of a square matrix $X \in \mathbb{R}^{n \times n}$ if

$$Xv = \lambda v, v \neq 0$$

- The rank $r \leq n$ of a matrix is the number of linearly independent rows/columns, which is also equal to the number of nonzero eigenvalues.
- $X \in \mathbb{R}^{n \times n}$ is **diagonalizable** if it has n linearly independent eigenvectors $v_1 \cdots v_n$.
- If $X \in \mathbb{R}^{n \times n}$ is diagonalizable, it admits an **Eigen-Value Decomposition (EVD)**

$$X[v_1 \cdots v_n] = [v_1 \cdots v_n] \begin{bmatrix} \lambda_1 & \cdots & 0 & 0 & \cdots & 0 \\ \vdots & \ddots & \vdots & \vdots & \ddots & \vdots \\ 0 & \cdots & \lambda_r & 0 & \cdots & 0 \\ 0 & \cdots & 0 & 0 & \cdots & 0 \\ \vdots & \ddots & \vdots & \vdots & \ddots & \vdots \\ 0 & \cdots & 0 & 0 & \cdots & 0 \end{bmatrix} \Rightarrow XV = V\Lambda \Rightarrow X = V\Lambda V^{-1}$$

Matrix Properties

- The trace of $\mathbf{X}$, $\text{tr}(\mathbf{X}) = \sum_i \mathbf{X}_{ii}$, satisfies the cyclic property: $\text{tr}(\mathbf{AB}) = \text{tr}(\mathbf{BA})$.
- The determinant $\det(\mathbf{X}) = \sum_{\sigma \in S_n} \text{sgn}(\sigma) \prod_i \mathbf{X}_{i,\sigma(i)}$ satisfies the cyclic property:

$$\det(\mathbf{AB}) = \det(\mathbf{A})\det(\mathbf{B}) = \det(\mathbf{BA}).$$

- **Spectral theorem:** For a symmetric matrix $\mathbf{S} \in \mathbb{R}^{d \times d}$, we always have that

$$\mathbf{S} = \mathbf{U}\mathbf{\Lambda}\mathbf{U}^\top$$

with **orthogonal** $\mathbf{U} = [\underbrace{u_1 \ u_2 \ \dots \ u_d}_{\text{Eigenvectors}}] \in \mathbb{R}^{d \times d}$ and **real** $\mathbf{\Lambda} = \text{diag}(\underbrace{\lambda_1, \dots, \lambda_d}_{\text{Eigenvalues}})$.

Inverse, Trace, Determinant, Positive Definiteness

- The inverse of a matrix $\mathbf{X} \in \mathbb{R}^{n \times n}$, if it exists, is the matrix $\mathbf{X}^{-1} \in \mathbb{R}^{n \times n}$ such that

$$\mathbf{X}\mathbf{X}^{-1} = \mathbf{X}^{-1}\mathbf{X} = \mathbf{I}$$

- The **trace/determinant** of a matrix is the sum/product of its eigenvalues

$$\text{trace}(\mathbf{X}) = \sum \lambda_i, \quad \det(\mathbf{X}) = \prod \lambda_i$$

- A matrix $\mathbf{X}$ is **positive (semi) definite** if it is symmetric, i.e., $\mathbf{X} = \mathbf{X}^T$, and if

$$v^T \mathbf{X} v > 0 \quad (v^T \mathbf{X} v \geq 0) \quad \text{for all } v \neq 0$$

- A matrix $\mathbf{X}$ is **positive (semi) definite** if its eigenvalues are positive (nonnegative)

$$\lambda_i > 0 \quad (\lambda_i \geq 0)$$

Singular Value Decomposition (SVD)

- Any matrix $\mathbf{X} \in \mathbb{R}^{m \times n}$ of rank r can be decomposed as

$$\mathbf{X} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^\top = [u_1 \cdots u_r u_{r+1} \cdots u_m] \begin{bmatrix} \sigma_1 & \cdots & 0 & 0 & \cdots & 0 \\ \vdots & \ddots & \vdots & \vdots & \ddots & \vdots \\ 0 & \cdots & \sigma_r & 0 & \cdots & 0 \\ 0 & \cdots & 0 & 0 & \cdots & 0 \\ \vdots & \ddots & \vdots & \vdots & \ddots & \vdots \\ 0 & \cdots & 0 & 0 & \cdots & 0 \end{bmatrix} [v_1 \cdots v_r v_{r+1} \cdots v_m]^\top$$

- $\mathbf{\Sigma} \in \mathbb{R}^{m \times n}$ is a “diagonal” matrix with r singular values $\sigma_1 \geq \sigma_2 \geq \cdots \geq \sigma_r > 0$
- $\mathbf{U} \in \mathbb{R}^{m \times m}$ and $\mathbf{V} \in \mathbb{R}^{n \times n}$ are orthonormal matrices of left/right singular vectors
- Orthonormal means that columns are unit norm and orthogonal to each other:

$$\mathbf{U}^\top \mathbf{U} = \mathbf{U} \mathbf{U}^\top = \mathbf{I}_{m \times m},$$

$$\mathbf{V}^\top \mathbf{V} = \mathbf{V} \mathbf{V}^\top = \mathbf{I}_{n \times n}$$

Compact Singular Value Decomposition (SVD)

- A matrix $\mathbf{X} \in \mathbb{R}^{m \times n}$ of rank r can be decomposed as

$$\begin{aligned}\mathbf{X} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^\top &= [u_1 \cdots u_r u_{r+1} \cdots u_m] \begin{bmatrix} \sigma_1 & \cdots & 0 & 0 & \cdots & 0 \\ \vdots & \ddots & \vdots & \vdots & \ddots & \vdots \\ 0 & \cdots & \sigma_r & 0 & \cdots & 0 \\ 0 & \cdots & 0 & 0 & \cdots & 0 \\ \vdots & \ddots & \vdots & \vdots & \ddots & \vdots \\ 0 & \cdots & 0 & 0 & \cdots & 0 \end{bmatrix} [v_1 \cdots v_r v_{r+1} \cdots v_m]^\top \\ &= \mathbf{U}_1 \mathbf{\Sigma}_1 \mathbf{V}_1^\top = [u_1 \cdots u_r] \begin{bmatrix} \sigma_1 & \cdots & 0 \\ \vdots & \ddots & \vdots \\ 0 & \cdots & \sigma_r \end{bmatrix} [v_1 \cdots v_r]^\top\end{aligned}$$

- $\mathbf{U}_1 \in \mathbb{R}^{m \times r}$ and $\mathbf{V}_1 \in \mathbb{R}^{n \times r}$ are orthonormal matrices of left/right singular vectors
- $\mathbf{\Sigma}_1 \in \mathbb{R}^{r \times r}$ is a diagonal matrix with r singular values $\sigma_1 \geq \sigma_2 \geq \cdots \geq \sigma_r > 0$

SVD Properties: Inverses

- We can use any partition of the SVD of a matrix to expand it as

$$X = U\Sigma V^T = [U_1 \quad U_2] \begin{bmatrix} \Sigma_1 & \mathbf{0} \\ \mathbf{0} & \Sigma_2 \end{bmatrix} [V_1 \quad V_2]^T = U_1 \Sigma_1 V_1^T + U_2 \Sigma_2 V_2^T$$

- **Example:** $I = U_1 U_1^T + U_2 U_2^T$, where $U_1^T U_1 = I$, $U_1^T U_2 = \mathbf{0}$, and $U_2^T U_2 = I$
- If X is invertible, we can use its SVD to compute the **inverse of a matrix**

$$X = U\Sigma V^T \Rightarrow X^{-1} = V\Sigma^{-1} U^T$$

- We can use any partition of the SVD of a matrix to compute its inverse as

$$X = U_1 \Sigma_1 V_1^T + U_2 \Sigma_2 V_2^T \Rightarrow X^{-1} = V_1 \Sigma_1^{-1} U_1^T + V_2 \Sigma_2^{-1} U_2^T$$

SVD Properties: Trace and Determinants

- We can use the SVD of a square matrix to compute its trace and determinant

$$\mathbf{X} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^\top \Rightarrow \text{tr}(\mathbf{X}) = \text{tr}(\mathbf{\Sigma}) = \sum_i \sigma_i \quad \text{and} \quad \det(\mathbf{X}) = \det(\mathbf{\Sigma}) = \prod_i \sigma_i$$

- We can use any partition of the SVD to compute its trace and determinant

$$\mathbf{X} = \mathbf{U}_1\mathbf{\Sigma}_1\mathbf{V}_1^\top + \mathbf{U}_2\mathbf{\Sigma}_2\mathbf{V}_2^\top \Rightarrow \text{tr}(\mathbf{X}) = \text{tr}(\mathbf{\Sigma}_1) + \text{tr}(\mathbf{\Sigma}_2)$$

$$\mathbf{X} = \mathbf{U}_1\mathbf{\Sigma}_1\mathbf{V}_1^\top + \mathbf{U}_2\mathbf{\Sigma}_2\mathbf{V}_2^\top \Rightarrow \det(\mathbf{X}) = \det(\mathbf{\Sigma}_1)\det(\mathbf{\Sigma}_2)$$

- Example

$$\mathbf{X} = \begin{bmatrix} \mathbf{I}_m & 0 \\ 0 & \sigma^2 \mathbf{I}_n \end{bmatrix} \Rightarrow \text{trace}(\mathbf{X}) = m + \sigma^2 n \quad \text{and} \quad \det(\mathbf{X}) = \sigma^{2n}$$

Outline

- Basics of Probability, Statistics, Information Theory
 - Discrete and Continuous Distributions, Independence
 - Marginals, Conditionals, Product Rule, Bayes Rule & Examples for Gaussians
 - Expectations, Covariance, Entropy, KL Divergence, Mutual Information
- Basics of Linear Algebra
 - Matrix, eigen-vectors, eigen-values, rank, eigen-value decomposition
 - Matrix properties: inverse, trace, determinant, positive definiteness
 - Singular value decomposition (SVD), compact SVD
 - SVD properties: inverse, trace, determinant
- **Basics of Optimization**
 - Gradient, Hessian, first and second order conditions for minima/maxima,
 - Gradient and stochastic gradient descent, alternating minimization, Lagrange Multipliers
 - Matrix-valued derivatives, Least squares

Unconstrained Optimization

- We need to minimize a **twice-differentiable function** $g(\theta)$ w.r.t. to a vector θ :

$$\min_{\theta} g(\theta)$$

- Step 1: Find a critical point where the gradient vector $\nabla_{\theta} g(\theta) = \mathbf{0}$.
- Step 2: Check if the critical point is
 - **A minimum:** The Hessian matrix $\nabla_{\theta}^2 g(\theta) \succcurlyeq 0$ (positive semi-definite), i.e., eigenvalues of $\nabla_{\theta}^2 g(\theta)$ are nonnegative.
 - **A maximum:** $\nabla_{\theta}^2 g(\theta) \preccurlyeq 0$ (negative semi-definite), i.e., all eigenvalues of $\nabla_{\theta}^2 g(\theta)$ are nonpositive.
 - **A saddle point:** Some eigenvalues of $\nabla_{\theta}^2 g(\theta)$ are positive, while others are negative.
- If it is intractable to calculate either of these steps directly (e.g., training a deep network), we need a tractable algorithm to find a minimum.

Optimization Algorithm: Gradient Descent

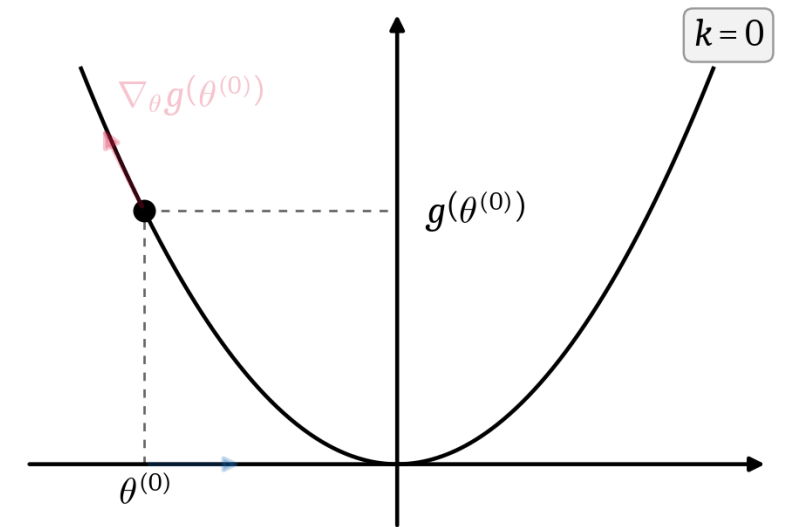
- **Goal:** optimize an objective that contains an expectation

$$\min_{\theta} g(\theta) := \mathbb{E}_{x \sim p}[f(x, \theta)]$$

- **Gradient descent:** first-order algorithm to optimize $g(\theta)$
 - Tractable even when θ is in high dimensions
 - Gradient descent: $\theta^{(k+1)} = \theta^{(k)} - \eta \nabla_{\theta} g(\theta^{(k)})$
 - Many variants to accelerate/deal with non-differentiability
- **Convergence:** If g is μ -strongly convex and L -smooth, and the step is small enough, $0 < \eta < 1/L$, then

$$||\theta^{(k)} - \theta^*||^2 \leq (1 - \eta\mu)^2 ||\theta^{(0)} - \theta^*||^2$$

- That is, gradient descent converges exponentially fast for strongly convex and smooth functions



Optimization Algorithm: Stochastic Gradient Descent

- **Challenge:** It is difficult to compute $\nabla_{\theta} g(\theta)$ in closed form
 - $\nabla_{\theta} g(\theta) = \nabla_{\theta} \mathbb{E}_{x \sim p}[f(x, \theta)] = \mathbb{E}_{x \sim p}[\nabla_{\theta} f(x, \theta)]$
 - Derivative is a linear operation, so we can often exchange derivative & expected value.
 - Often p is the true data distribution which we do not know; we have samples from p
 - Even if we knew p , computing the integral is potentially very complicated f is difficult
- **Solution:** Approximate $\nabla_{\theta} g(\theta)$ with samples
 - Let $x_1, \dots, x_b$ be a batch of i.i.d. samples from p
 - $\frac{1}{b} \sum_i^b \nabla_{\theta} f(x_i, \theta)$ is an unbiased estimator of $\nabla_{\theta} g(\theta)$
 - Stochastic gradient descent: $\theta^{(k+1)} = \theta^{(k)} - \eta \frac{1}{b} \sum_i^b \nabla_{\theta} f(x_i, \theta)$

Constrained Minimization: Lagrange Multipliers

- Constrained problem: $\min g(\theta)$ subject to $h(\theta) = 0$.
- We can convert this into an unconstrained objective called the Lagrangian

$$L(\theta, \lambda) = g(\theta) + \lambda h(\theta)$$

- We then calculate the critical point of the Lagrangian

$$\frac{dL}{d\theta} = \nabla g(\theta) + \lambda \nabla h(\theta) = 0, \quad \frac{dL}{d\lambda} = h(\theta) = 0$$

- At a constrained optimum θ^* , there must exist a Lagrange multiplier λ^* such that

$$\nabla g(\theta^*) + \lambda^* \nabla h(\theta^*) = 0, \quad h(\theta^*) = 0$$

- We can then check the second-order condition at this critical point to see if it is a minimum.

Matrix-valued derivatives

- We usually need to differentiate a scalar function with respect to a matrix.
 - Example: Differentiate a loss function with respect to the weights of a layer in a neural network.
- For scalar $g(\mathbf{X})$, the gradient has the same shape as $\mathbf{X}$. Examples:

$$\frac{d}{d\mathbf{X}} \text{trace}(\mathbf{A}\mathbf{X}) = \mathbf{A}^\top$$

$$\frac{d}{d\mathbf{X}} \log(\det(\mathbf{X})) = \mathbf{X}^{-\top}$$

Least squares

- Fit a linear model by minimizing squared error $\min_x ||\mathbf{Ax} - \mathbf{b}||^2$, where $\mathbf{A} \in \mathbb{R}^{N \times d}$, $\mathbf{x} \in \mathbb{R}^d$, $\mathbf{b} \in \mathbb{R}^N$

$$\frac{d}{d\mathbf{x}} ||\mathbf{Ax} - \mathbf{b}||^2 = 2\mathbf{A}^\top (\mathbf{Ax} - \mathbf{b}) = \mathbf{0} \Rightarrow \mathbf{A}^\top \mathbf{A} \mathbf{x} = \mathbf{A}^\top \mathbf{b}$$

- Three regimes depend on the number of observations N , parameters d , and $\mathbf{A}$'s rank r
 - $N > d$: overdetermined. **Unique least-square solution** if $\mathbf{A}$ has full column rank ($r = d$)

$$\mathbf{x}^* = (\mathbf{A}^\top \mathbf{A})^{-1} \mathbf{A}^\top \mathbf{b}.$$

- $N = d$: square system. **Unique solution** if $\mathbf{A}$ is invertible ($r = d = N$): $\mathbf{x}^* = \mathbf{A}^{-1} \mathbf{b}$
- $N < d$: underdetermined. **Unique minimum-norm** (smallest $||\mathbf{x}||_2$) solution if $\mathbf{A}$ has full row rank ($r = N$):

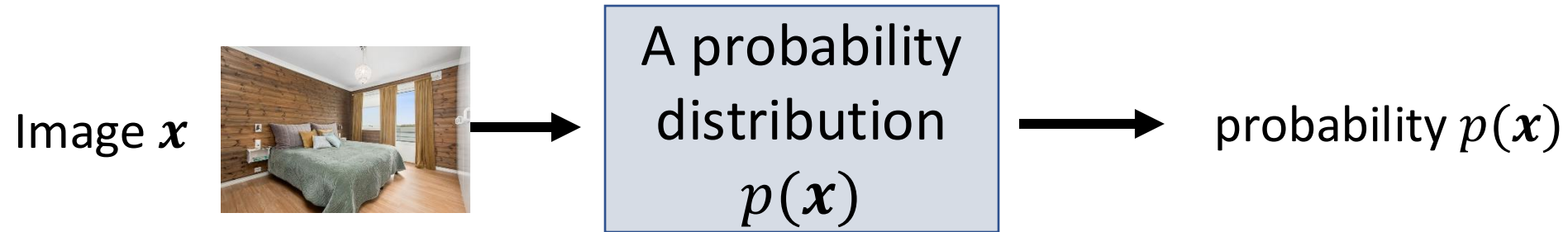
$$\mathbf{x}^* = \mathbf{A}^\top (\mathbf{A} \mathbf{A}^\top)^{-1} \mathbf{b}.$$

Outline

- **Generative vs Discriminative Models**
- Learning Generative Models
 - Learning Criterion: Maximum Likelihood Estimation
 - Learning Algorithm: Stochastic Gradient Descent
- Classes of Generative Models
 - Gaussian Models: Closed form Solution
 - General Models: Need for Structure
 - Taxonomy of Models
 - Latent variable models, Autoregressive models, Energy based models

Statistical Generative Models

- A statistical generative model is a probability distribution $p(\mathbf{x})$



- It is generative because **sampling from $p(\mathbf{x})$ generates new images**



...



Discriminative vs. Generative Models

Discriminative: classify bedroom vs. dining room

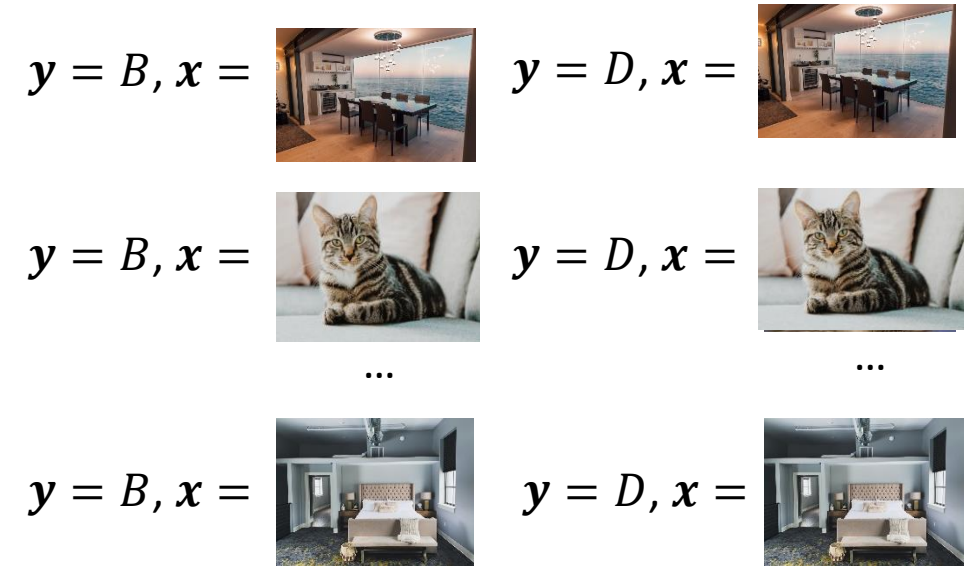


The image x is given. **Goal:** decision boundary, via **conditional distribution over label y**

$$p(y = \text{Bedroom} \mid x = \text{dining room image}) = 0.0001$$

Ex: logistic regression, convolutional net, etc.

Generative: generate x



The input x is **not** given. Requires a model of the **joint distribution over both x and y**

$$p(y = \text{Bedroom}, x = \text{dining room image}) = 0.0002$$

Discriminative vs. Generative

Joint and conditional are related via **Bayes Rule**:

$$p(\mathbf{y} = \text{Bedroom} \mid \mathbf{x} = \text{img}) = \frac{p(\mathbf{y} = \text{Bedroom}, \mathbf{x} = \text{img})}{p(\mathbf{x} = \text{img})}$$

Discriminative: $\mathbf{y}$ is simple; $\mathbf{x}$ is always given, so not need to model

$$p(\mathbf{x} = \text{img})$$

Thus, a discriminative model cannot handle missing data $p(\mathbf{y} = \text{Bedroom} \mid \mathbf{x} = \text{img})$

Conditional Generative Models

Class **conditional generative models** are also possible:

$$p(\boldsymbol{x} = \img alt="A modern dining room with a black table and 6 chairs, overlooking the ocean." data-bbox="398 324 494 437" | \boldsymbol{y} = \text{Bedroom})$$

It's often useful to condition on rich side information $\boldsymbol{Y}$

$$p(\boldsymbol{x} = \img alt="A modern dining room with a black table and 6 chairs, overlooking the ocean." data-bbox="260 541 356 653" | \text{Caption} = \text{"A black table with 6 chairs"})$$

A discriminative model is a very simple conditional generative model of $\boldsymbol{y}$:

$$p(\boldsymbol{y} = \text{Bedroom} | \boldsymbol{x} = \img alt="A modern dining room with a black table and 6 chairs, overlooking the ocean." data-bbox="553 763 649 876")$$

Why Generative Models?

- AI Is Not Only About Decision Making

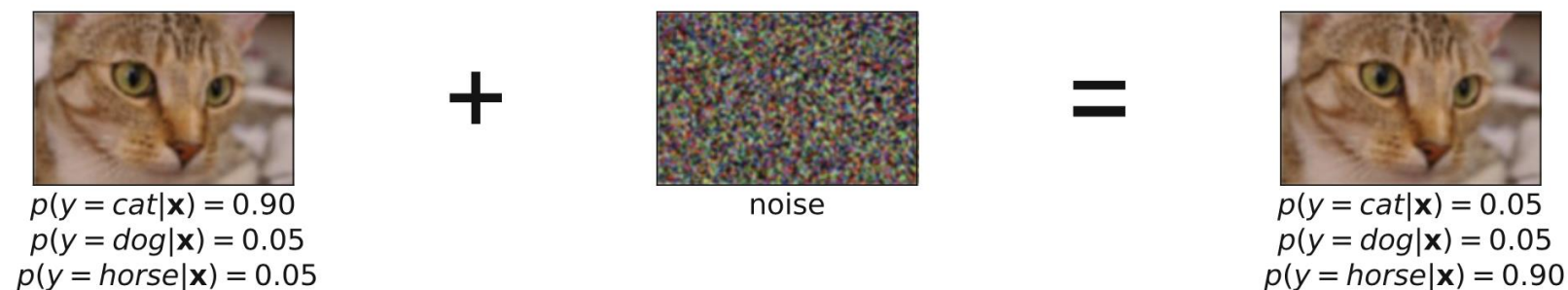


Fig. 1.1 An example of adding noise to an almost perfectly classified image that results in a shift of predicted label

- Importance of uncertainty and understanding in decision making

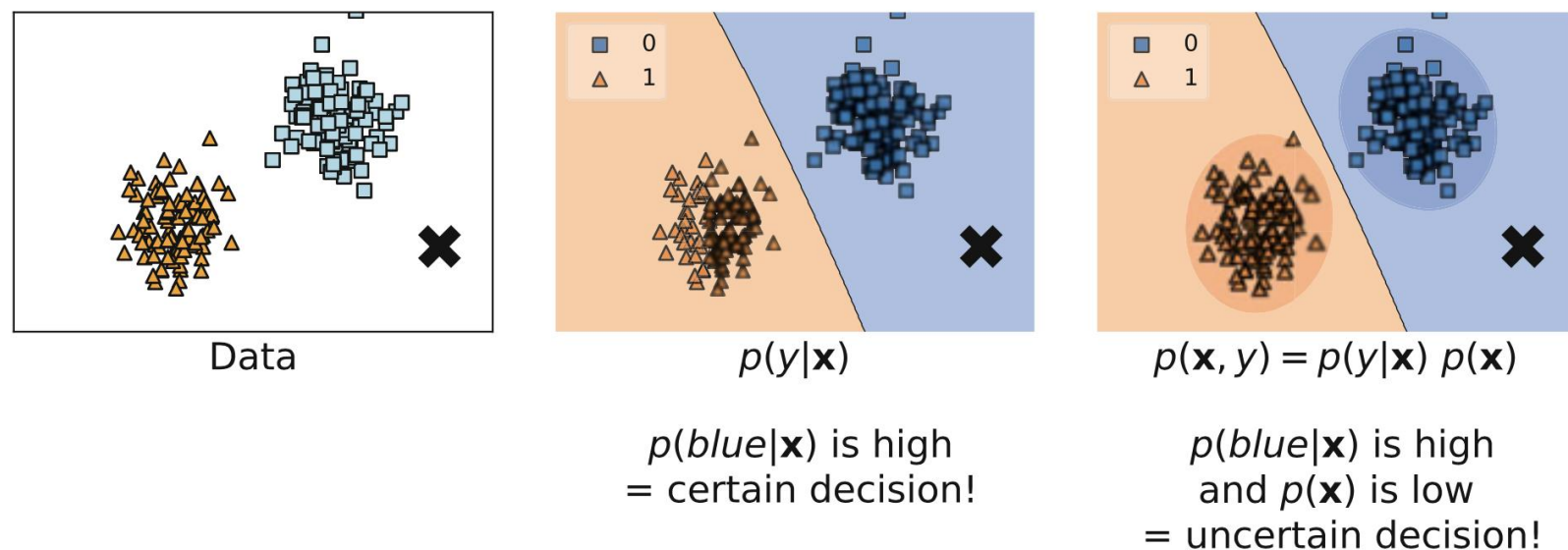


Fig. 1.2 An example of data (left) and two approaches to decision making: (middle) a discriminative approach and (right) a generative approach

Outline

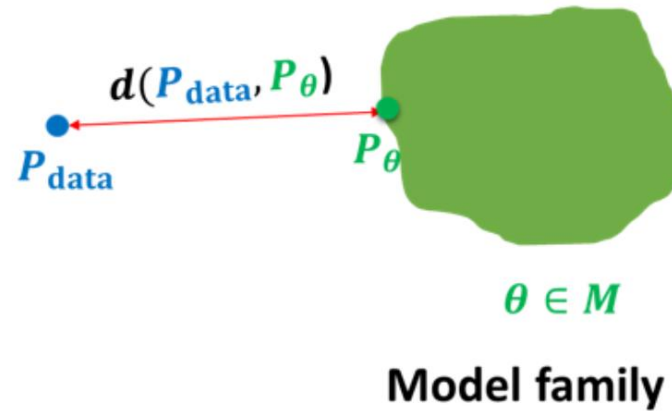
- Generative vs Discriminative Models
- **Learning Generative Models**
 - Learning Criterion: Maximum Likelihood Estimation
 - Learning Algorithm: Stochastic Gradient Descent
- Classes of Generative Models
 - Gaussian Models: Closed form Solution
 - General Models: Need for Structure
 - Taxonomy of Models
 - Latent variable models, Autoregressive models, Energy based models

Learning Generative Models

- We are given a training set of examples, e.g., images of dogs



$$x_i \sim P_{\text{data}} \\ i = 1, 2, \dots, n$$



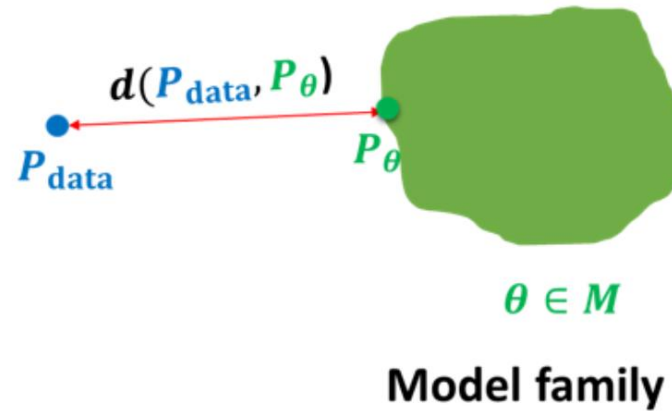
- We want to learn a probability distribution $p_{\theta}(x)$ over images x to allow for
 - **Generation:** If we sample $x_{\text{new}} \sim p_{\theta}(x)$, then x_{new} should look like a dog (sampling)
 - **Density estimation:** $p_{\theta}(x)$ should be high if x looks like a dog, and low otherwise (anomaly detection)
 - **Unsupervised representation learning:** We should be able to learn what these images have in common, e.g., ears, tail, etc. (features)

Learning Generative Models

- We are given a training set of examples, e.g., images of dogs



$$x_i \sim P_{\text{data}} \\ i = 1, 2, \dots, n$$



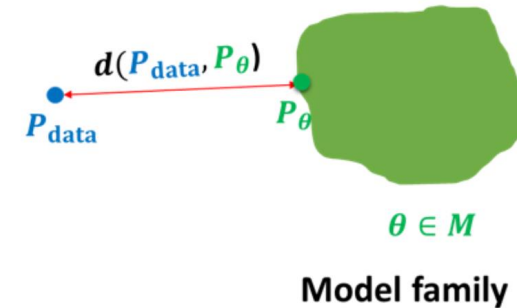
- We want to learn a probability distribution $p_{\theta}(x)$ over images x
- What learning criterion should we use?
- What optimization algorithm should we use?
- What classes of models should we learn?

Learning Criterion: Maximum Likelihood Estimation

- **Goal:** given a dataset $\mathcal{D} = \{\mathbf{x}_1, \dots, \mathbf{x}_N\}$ of i.i.d. samples from the unknown data distribution $p_{\text{data}}(\mathbf{x})$, we want to learn a distribution $p_{\theta}(\mathbf{x})$ that is as “close” as possible to $p_{\text{data}}(\mathbf{x})$



$\mathbf{x}_i \sim P_{\text{data}}$
 $i = 1, 2, \dots, n$



- Taking d as the KL divergence introduced before: $\min_{\theta} KL[p_{\text{data}}(\mathbf{x}) || p_{\theta}(\mathbf{x})]$
- Since $KL[p_{\text{data}}(x) || p_{\theta}(x)] = \mathbb{E}_{x \sim p_{\text{data}}} \left[\log \frac{p_{\text{data}}(x)}{p_{\theta}(x)} \right]$ and we optimize over θ , the above problem is equivalent to $\max_{\theta} \mathbb{E}_{x \sim p_{\text{data}}} [\log p_{\theta}(x)]$
- As we do not know the true distribution $p_{\text{data}}(x)$ and only have samples $\mathcal{D}$ from it, we can replace the above objective with an unbiased estimate of it

$$\max_{\theta} \frac{1}{N} \sum_{i=1}^N \log p_{\theta}(x_i)$$

This is the classic Maximum Likelihood Estimation (MLE) principle!

Maximum Likelihood Estimation (MLE)

- Likelihood is expressed as the joint distribution over all samples
- And by our i.i.d. assumption

$$\mathcal{L}(\theta) = p_{\theta}(\mathbf{x}_1, \dots, \mathbf{x}_N) = \prod_{i=1}^N p_{\theta}(\mathbf{x}_i)$$

- Taking the log, we can rewrite

$$\ell(\theta) = \log(\mathcal{L}(\theta)) = \log\left(\prod_{i=1}^N p_{\theta}(\mathbf{x}_i)\right) = \sum_{i=1}^N \log p_{\theta}(\mathbf{x}_i)$$

- The maximum likelihood estimator is the parameters that maximizes $\ell(\theta)$, i.e.

$$\hat{\theta}_{ML} = \operatorname{argmax}_{\theta} \sum_{i=1}^N \log p_{\theta}(\mathbf{x}_i)$$

Outline

- Generative vs Discriminative Models
- Learning Generative Models
 - Learning Criterion: Maximum Likelihood Estimation
 - Learning Algorithm: Stochastic Gradient Descent
- **Classes of Generative Models**
 - Gaussian Models: Closed form Solution
 - General Models: Need for Structure
 - Taxonomy of Models
 - Latent variable models, Autoregressive models, Energy based models

Gaussian Parameter Estimation via MLE

- Given: N i.i.d. samples $\mathbf{x}_1, \dots, \mathbf{x}_N$ from an unknown Gaussian $\mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma})$ in $\mathbb{R}^D$
- Goal: use MLE to estimate the parameters $\theta = (\boldsymbol{\mu}, \boldsymbol{\Sigma})$ of the Gaussian distribution
- Recall Gaussian density: $p(\mathbf{x}) = \frac{1}{\sqrt{(2\pi)^D \det(\boldsymbol{\Sigma})}} \exp\left(-\frac{1}{2}(\mathbf{x} - \boldsymbol{\mu})^\top \boldsymbol{\Sigma}^{-1}(\mathbf{x} - \boldsymbol{\mu})\right)$
- This allows us to write down the likelihood function...

$$\mathcal{L}(\theta) = \prod_{i=1}^N p_{\theta}(\mathbf{x}_i) = \frac{\exp\left(-\frac{1}{2} \sum_{i=1}^N (\mathbf{x}_i - \boldsymbol{\mu})^\top \boldsymbol{\Sigma}^{-1}(\mathbf{x}_i - \boldsymbol{\mu})\right)}{(2\pi)^{\frac{ND}{2}} \det(\boldsymbol{\Sigma})^{\frac{N}{2}}}$$

- ... and the log of the likelihood

$$\begin{aligned} \ell(\theta) &= \sum_{i=1}^N -\frac{D}{2} \log(2\pi) - \frac{1}{2} \log(\det(\boldsymbol{\Sigma})) - (\mathbf{x}_i - \boldsymbol{\mu})^\top \boldsymbol{\Sigma}^{-1}(\mathbf{x}_i - \boldsymbol{\mu}) \\ &= -\frac{ND}{2} \log(2\pi) - \frac{N}{2} \log(\det(\boldsymbol{\Sigma})) - \frac{1}{2} \sum_{i=1}^N (\mathbf{x}_i - \boldsymbol{\mu})^\top \boldsymbol{\Sigma}^{-1}(\mathbf{x}_i - \boldsymbol{\mu}) \end{aligned}$$

Optimizing the Log Likelihood

- Reminder: Log-likelihood objective

$$\ell(\theta) = -\frac{ND}{2} \log(2\pi) - \frac{N}{2} \log(\det(\mathbf{\Sigma})) - \frac{1}{2} \sum_{i=1}^N (\mathbf{x}_i - \boldsymbol{\mu})^\top \mathbf{\Sigma}^{-1} (\mathbf{x}_i - \boldsymbol{\mu})$$

- To find the optimal θ , we take the derivatives of $\ell(\theta)$ and set them to 0

$$\frac{\partial \ell(\theta)}{\partial \boldsymbol{\mu}} = 0, \quad \frac{\partial \ell(\theta)}{\partial \mathbf{\Sigma}} = 0$$

- Taking the derivative of the log-likelihood w.r.t. to the mean yields

$$\frac{\partial \ell(\theta)}{\partial \boldsymbol{\mu}} = \sum_{i=1}^N \mathbf{\Sigma}^{-1} (\mathbf{x}_i - \boldsymbol{\mu}) = 0 \Rightarrow \sum_{i=1}^N (\mathbf{x}_i - \boldsymbol{\mu}) = 0 \Rightarrow \hat{\boldsymbol{\mu}}_{ML} = \frac{1}{N} \sum_{i=1}^N \mathbf{x}_i$$

Optimizing the Log Likelihood

- Reminder: Log-likelihood objective

$$\ell(\theta) = -\frac{ND}{2} \log(2\pi) - \frac{N}{2} \log(\det(\mathbf{\Sigma})) - \frac{1}{2} \sum_{i=1}^N (\mathbf{x}_i - \boldsymbol{\mu})^\top \mathbf{\Sigma}^{-1} (\mathbf{x}_i - \boldsymbol{\mu})$$

- Before taking the derivative, we make a change of variables to handle the inverse covariance $\mathbf{S} = \mathbf{\Sigma}^{-1}$ and note the following identity involving traces

$$\mathbf{x}^\top \mathbf{S} \mathbf{x} = \text{tr}(\mathbf{x}^\top \mathbf{S} \mathbf{x}) = \text{tr}(\mathbf{S} \mathbf{x} \mathbf{x}^\top)$$

- We can then rewrite the log-likelihood in terms of $\mathbf{S}$

$$\ell(\theta) = -\frac{ND}{2} \log(2\pi) - \frac{N}{2} \log \det(\mathbf{S}^{-1}) - \frac{1}{2} \text{tr} \left(\mathbf{S} \sum_{i=1}^N (\mathbf{x}_i - \boldsymbol{\mu})(\mathbf{x}_i - \boldsymbol{\mu})^\top \right)$$

Optimizing the Log Likelihood

- From our re-written log-likelihood function

$$\ell(\theta) = -\frac{ND}{2} \log(2\pi) + \frac{N}{2} \log(\det(\mathbf{S})) - \frac{1}{2} \text{tr} \left(\mathbf{S} \sum_{i=1}^N (\mathbf{x}_i - \boldsymbol{\mu})(\mathbf{x}_i - \boldsymbol{\mu})^\top \right)$$

- Taking the derivative with respect to $\mathbf{S}$

$$\frac{\partial \ell(\theta)}{\partial \mathbf{S}} = \frac{N}{2} \mathbf{S}^{-1} - \frac{1}{2} \sum_{i=1}^N (\mathbf{x}_i - \boldsymbol{\mu})(\mathbf{x}_i - \boldsymbol{\mu})^\top = 0$$

- Arriving at our desired ML estimator for the covariance

$$\hat{\boldsymbol{\Sigma}}_{ML} = \mathbf{S}^{-1} = \frac{1}{N} \sum_{i=1}^N (\mathbf{x}_i - \boldsymbol{\mu})(\mathbf{x}_i - \boldsymbol{\mu})^\top$$

ML Estimators for Mean and Variance of a Gaussian

- **In summary:** Given N i.i.d. samples $\mathbf{x}_1, \dots, \mathbf{x}_N$ from an unknown Gaussian $\mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma})$ in $\mathbb{R}^D$
- The log-likelihood function is given by

$$\ell(\theta) = -\frac{ND}{2} \log(2\pi) - \frac{N}{2} \log(\det(\boldsymbol{\Sigma})) - \frac{1}{2} \sum_{i=1}^N (\mathbf{x}_i - \boldsymbol{\mu})^\top \boldsymbol{\Sigma}^{-1} (\mathbf{x}_i - \boldsymbol{\mu})$$

- The maximum log likelihood estimators for the mean and covariance are

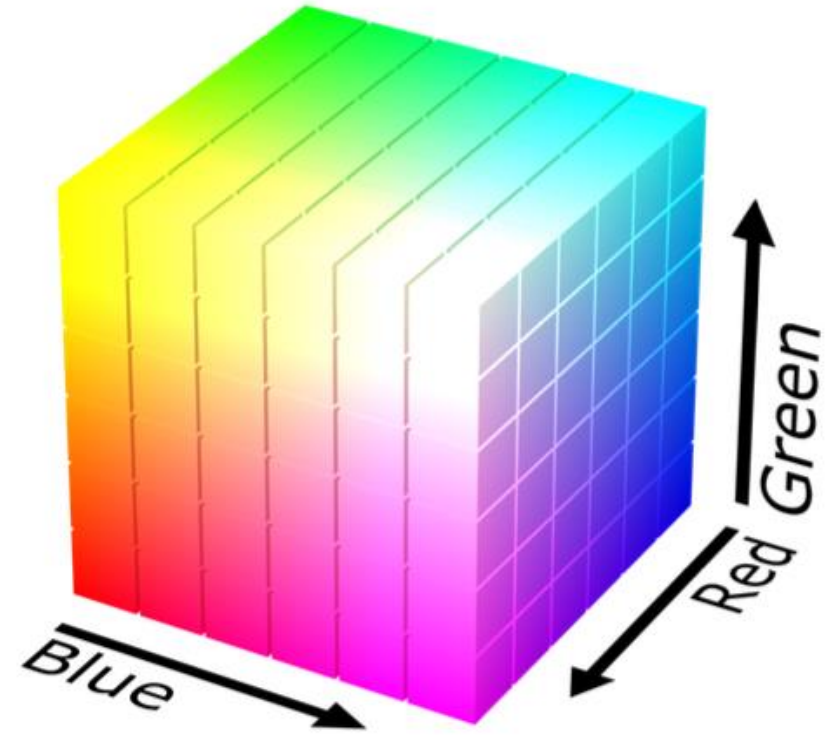
$$\hat{\boldsymbol{\mu}}_{ML} = \frac{1}{N} \sum_{i=1}^N \mathbf{x}_i \quad \hat{\boldsymbol{\Sigma}}_{ML} = \frac{1}{N} \sum_{i=1}^N (\mathbf{x}_i - \boldsymbol{\mu})(\mathbf{x}_i - \boldsymbol{\mu})^\top$$

Outline

- Basics of Probability, Statistics, Information Theory
 - Discrete and Continuous Distributions, Independence
 - Marginals, Conditionals & Example for a Gaussian
 - Entropy, Mutual Information, KL Divergence
- Generative vs Discriminative Models
- Learning Generative Models
 - Learning Criterion: Maximum Likelihood Estimation
 - Learning Algorithm: Stochastic Gradient Descent
- Classes of Generative Models
 - Gaussian Models: Closed form Solution
 - **General Models: Need for Structure**
 - Taxonomy of Models
 - Latent variable models, Autoregressive models, Energy based models

Example: RGB images

- To model a single pixel's color, one needs three discrete random variables:
 - Red Channel R taking values in $\{0, \dots, 255\}$
 - Green Channel G taking values in $\{0, \dots, 255\}$
 - Blue Channel B taking values in $\{0, \dots, 255\}$
- Sampling from the joint distribution $(r, g, b) \sim p(R, G, B)$ randomly generates a color for the pixel. How many parameters do we need to specify the joint distribution $p(R = r, G = g, B = b)$?



$$256 * 256 * 256 - 1$$

Example: Joint Distribution



- Suppose $X_1, \dots, X_n$ are Bernoulli random variables modelling n pixels of an image
- How many possible states?

$$\underbrace{2 \times 2 \times \dots \times 2}_{n \text{ times}} = 2^n$$

- Sampling from $p(x_1, \dots, x_n)$ generates an image
- How many parameters to specify the joint distribution $p(x_1, \dots, x_n)$ over n binary pixels?

$$2^n - 1$$

Structure Through Independence

- If $X_1, \dots, X_n$ are independent, then
$$p(x_1, \dots, x_n) = p(x_1)p(x_2) \cdots p(x_n)$$
- How many possible states? 2^n
- How many parameters to specify the joint distribution $p(x_1, \dots, x_n)$?
 - How many to specify the marginal distribution $p(x_1)$? 1
- 2^n entries can be described by just n numbers (if each X_i just take 2 values)!
- Independence assumption is too strong. Model not likely to be useful
 - For example, each pixel chosen independently when we sample from it.



Structure Through Conditional Independence

- Using Chain Rule

$$p(x_1, \dots, x_n) = p(x_1)p(x_2 \mid x_1)p(x_3 \mid x_1, x_2) \cdots p(x_n \mid x_1, \dots, x_{n-1})$$

- How many parameters? $1 + 2 + \dots + 2^{n-1} = 2^n - 1$
 - $p(x_1)$ requires 1 parameter
 - $p(x_2 \mid x_1 = 0)$ and $p(x_2 \mid x_1 = 1)$ requires 1 parameter each $\Rightarrow$ 2 parameters in total
 - $p(x_n \mid x_1, \dots, x_{n-1})$ requires 2^{n-1} parameters
 - $2^n - 1$ is still exponential, chain rule does not buy us anything.

- Now suppose the **Markov property** holds, i.e., $X_{i+1} \perp X_1, \dots, X_{i-1} \mid X_i$, then

$$\begin{aligned} p(x_1, \dots, x_n) &= p(x_1)p(x_2 \mid x_1)p(x_3 \mid \cancel{x_1}, x_2) \cdots p(x_n \mid \cancel{x_1, \dots}, x_{n-1}) \\ &= p(x_1)p(x_2 \mid x_1)p(x_3 \mid x_2) \cdots p(x_n \mid x_{n-1}) \end{aligned}$$

- How many parameters? $2n - 1$. Exponential reduction!

Taxonomy of Generative Models

- To effectively learn generative models we need to impose some structure

- Autoregressive Models

$$p(\mathbf{x}) = p(x_0) \prod_{i=1}^D p(x_i | \mathbf{x}_{<i}),$$

- Latent Variable Models

$$\begin{aligned} \mathbf{z} &\sim p(\mathbf{z}) \\ \mathbf{x} &\sim p(\mathbf{x} | \mathbf{z}) \end{aligned}$$

- Energy Based Models

$$p(\mathbf{x}) = \frac{\exp\{-E(\mathbf{x})\}}{Z}$$

Taxonomy of Generative Models

